SecretHandle: OS-Minted, Destination-Bound Opaque Secrets for Mobile AI Agents

Research concept document (not a paper draft)

MobiSys 2027 idea repository, Idea 10

Compiled 2026-10-03 from a web literature sweep; PROJECT_FOLDER contained no PDFs

1 Title

SecretHandle: OS-Minted, Destination-Bound Opaque Secrets for Mobile AI Agents.

2 Primary Target Venue

ACM MobiSys 2027. The contribution is an Android mechanism that lets an agent use secrets (OTPs, credentials, phone numbers, addresses, payment and account identifiers) through unforgeable, destination-bound handles minted at the authoritative OS source, with the raw value never entering the agent's observation or actuation, evaluated on real login/2FA/checkout flows. It is an OS data-isolation mechanism with on-device measurements.

3 Alternative Venues

- USENIX Security (`https://www.usenix.org/conferences`): secret handling and the agent threat model.
- IEEE S&P (`https://sp2027.ieee-security.org`): if the capability model and binding guarantee dominate.
- NDSS (`https://www.ndss-symposium.org`): agent-security track.
- ACM CCS (`https://www.sigsac.org/ccs.html`): secondary.

4 Research Problem

Can Android let an AI agent use sensitive values without observing them, by minting unforgeable opaque handles at the authoritative OS source (SMS/notification OTP pipeline, Credential Manager, clipboard, wallet) and resolving a handle to the raw value only when the agent targets a destination (app, field, origin) that the handle is bound to, so that neither the raw value nor a usable copy ever enters the agent's perception or a wrong destination?

5 Why Mobile?

The authoritative sources are mobile OS services: the SMS Retriever and User Consent APIs, Android 15's sensitive-notification OTP redaction, Credential Manager, the clipboard, and wallet

tokenization. The agent reaches the UI through accessibility, so an OTP it must enter is otherwise read off the screen (the 2510.27140 attack opens the Messages app and forwards the code). Destination binding uses Android's package and WebView-origin identity. On a server there is no OS SMS pipeline, no Credential Manager, no on-device wallet, and no UI to type into; the problem and its enforcement point are mobile.

6 Motivation

Real account-takeover via agents already happened: the Comet browser agent read an OTP from Gmail and posted it publicly (Brave, Aug 2025); 2510.27140 shows mobile agents forwarding Alipay codes under injection. Placeholder anonymization exists but mints handles by screen scraping: AutoDroid (MobiCom 2024) did prompt-side substitution; "Anonymization-Enhanced Privacy" (arXiv 2602.10139) resolves `type(PLACEHOLDER)` locally but relies on an NER/OCR detector, is session-scoped, "validates index ranges but not which field receives a resolved value", and never tests an OTP or login flow. Industry agentic-payment protocols (Google AP2, Stripe shared payment tokens, Visa/Mastercard agent tokens) tokenize payments server-side but do not cover OTP, phone, email, address, clipboard, or OS-level enforcement. Android's own Autofill and Credential Manager are app-initiated and require the consuming app to drive them; an accessibility agent cannot invoke them.

7 Closest Prior Work

- **Anonymization-Enhanced Privacy for Mobile GUI Agents** (arXiv 2602.10139): type-preserving placeholders, local resolution at `type` time, a gatekeeper SLM for bounded compute; detector-based, session-scoped, no destination binding, no OTP evaluation, honest-but-curious cloud only.
- **AutoDroid** (MobiCom 2024): earliest mobile placeholder scheme with reverse mapping.
- **CaMeL** (arXiv 2503.18813), **FIDES** (arXiv 2505.23643), **Dual LLM pattern** (2023): capabilities/symbolic references hide values from the planner in tool-calling settings.
- **AirGapAgent** (CCS 2024), **IsolateGPT** (NDSS 2025), **Progent** (arXiv 2504.11703): minimization and isolation for tool agents.
- **Prompt sanitizers**: HaS, Casper, Rescriber (CHI 2025), PAPILLON (NAACL 2025), PrivacyRestore (ACL 2025).
- **Attacks**: Comet OTP exfiltration (Brave 2025), 2510.27140 OTP forwarding, EIA (ICLR 2025), fine-print injection (arXiv 2504.11281).
- **Platform/industry**: SMS Retriever and User Consent APIs, Android 15 sensitive-notification OTP redaction, Credential Manager, Autofill (`SmsCodeAutofillClient`), clipboard `EXTRA_IS_SENSITIVE`, Protected Confirmation, StrongBox; Apple `oneTimeCode` AutoFill, passkeys; Google AP2, Stripe Shared Payment Tokens, Visa/Mastercard agent tokens; OAuth on-behalf-of and Cross-App Access drafts; MCP authorization (RFC 8707 audience binding).

8 Research Gap

No OS mechanism lets an untrusted agent use secrets that are not already on its screen while remaining blind to them, with destination binding and lifecycle. Missing: (1) handles minted at the authoritative source (SMS/notification pipeline, Credential Manager, clipboard, wallet), not by screen scraping, so security does not depend on detector recall; (2) binding a handle to the

app/origin and task that may consume it (as SMS Retriever hash-binds a message to an app), so an injected instruction cannot route an OTP to an attacker's field; (3) a typed lifecycle (single-use and TTL for OTPs, task-scoped for identifiers, confirm-per-use for payment); (4) observation-side redaction of a resolved value in the next screenshot and accessibility tree; (5) unforgeability against a malicious app rendering fake handle-like strings or fake OTP SMS.

9 Threat Model or Privacy Model

Trusted: system_server and the SecretHandle service, the OS source pipelines (SMS, notification classifier, Credential Manager, clipboard, wallet), hardware-backed key storage. **Untrusted**: the agent and its model (may be prompt-injected), third-party apps (one may spoof handle-like content or fake OTP SMS), cloud model endpoints, web content. **Attacker goals**: read a secret the agent handles; redirect a secret to an attacker-controlled destination; replay or reuse a secret; forge a handle. **Assets**: OTPs, credentials, phone/email/address, payment and account identifiers. **Goal**: the agent can issue `ENTER(handle)` into a bound destination and the OS resolves it locally; the raw value never enters the agent's observation or a non-bound destination; handles are unforgeable OS capabilities; resolved values are redacted from subsequent agent observation; single-use secrets cannot be reused. Utility goal: login, 2FA, and checkout flows complete.

10 Core Idea

A *secret handle* is an unforgeable capability object minted by the OS at the authoritative source: when an OTP arrives (SMS Retriever / sensitive-notification pipeline), when the user selects a credential (Credential Manager), when a value is copied (clipboard), or when a payment identifier is provisioned (wallet). The handle carries a type, a destination binding (allowed package and, for web, document origin; derived from the task and, where possible, from the source's own binding such as SMS Retriever's app hash), and a lifecycle (single-use plus TTL for OTPs, task-scoped for identifiers, confirm-per-use for payment). The agent sees only the handle in its accessibility tree and screenshot (never the value), reasons about it ("the code arrived, enter it into the login field"), and issues `ENTER(handle)` targeting a node; the OS resolves the handle to the raw value and injects it only if the target node's package/origin matches the binding, then redacts the value from the agent's subsequent observation of that field. Composes with ScreenCaps blind actuation (the agent acts on a node it cannot read) and with IntentGuard task tokens (the binding is part of the task capability).

11 Preliminary System Architecture

- **Handle Mint** (new system service; OS modification): hooks the SMS/notification OTP pipeline, Credential Manager, clipboard, and wallet to mint typed, bound, unforgeable handles; keys in StrongBox/Knox.
- **Resolver** (input-dispatch and accessibility action path modification): on `ENTER(handle)`, checks the target node's package/origin against the binding, injects the value, enforces single-use/TTL.
- **Observation Redactor** (AccessibilityManagerService and the agent's screenshot path): masks resolved values in the agent's view (extending Android 15 screenshare hiding).
- **Agent SDK** (application modification): exposes handles in the observation; `ENTER(handle)` action; adapters for open-source agents.

- **Hardware**: hardware-backed keys; Protected Confirmation for payment-class handles.

12 Novelty Boundary

What is fundamentally new: handles minted at the authoritative OS source rather than scraped from the screen (so security is independent of detector recall), destination-and-task binding that defeats injection-driven redirection even when the agent is fully compromised, a typed lifecycle, and observation-side redaction after entry. 2602.10139 and AutoDroid resolve text placeholders with no binding and no OTP support; CaMeL/Dual-LLM hide values from the planner in tool settings; Autofill/Credential Manager are app-initiated and trust the app; AP2/Stripe tokens are server-side and payment-only. **What would not exist otherwise**: an OS layer that makes secrets "usable but invisible" to untrusted agents across apps, with enforcement, and the first evaluation on real OTP/login/checkout flows. Novelty requires further literature verification.

13 Expected Contributions

1. An unforgeable, destination-bound, typed secret-handle abstraction minted at authoritative OS sources.
2. A resolver that injects values only into bound destinations, defeating injection-driven redirection.
3. A typed lifecycle (single-use/TTL/confirm-per-use) and observation-side redaction after entry.
4. An AOSP implementation across the OTP pipeline, Credential Manager, clipboard, and wallet.
5. An evaluation on real login, 2FA, and checkout flows against the Comet/2510.27140 attack patterns, measuring exposure to the model per task and task success.

14 Evaluation Plan

- **Platform**: AOSP on Pixel and a Samsung device for Knox-backed keys; two or three open test apps (login plus SMS OTP; checkout with a wallet token; clipboard-based 2FA) since no benchmark ships these flows.
- **Agents**: DroidRun, Mobile-Agent-v3, AppAgent, with cloud and on-device models.
- **Workloads**: AndroidWorld plus the OTP/login/checkout test apps; SPA-Bench for broader coverage.
- **Attacks**: 2510.27140 OTP forwarding; Comet-style read-and-exfiltrate; injection redirecting `ENTER` to an attacker field; a malicious app rendering fake handle strings and fake OTP SMS; post-entry screen re-read.
- **Baselines**: no protection (agent reads OTP off screen); 2602.10139 placeholders; Autofill where invocable.
- **Metrics**: secret values reaching the model per task (should be zero), redirection success under injection (should be zero), task success on login/2FA/checkout, post-entry leakage, handle-forgery resistance, resolver latency.
- **Ablations**: binding on/off; lifecycle on/off; observation redaction on/off.

15 Feasibility

Medium-high. The SMS Retriever/User Consent APIs, Credential Manager, clipboard, and Android 15 sensitive-notification pipeline are public and hookable in AOSP; StrongBox provides key storage; destination binding reuses package and WebView-origin identity. The main engineering is wiring the mint points and the resolver into input dispatch; the test apps are quick to build. Effort: two engineers, four to five months.

16 Potential Risks

- "2602.10139 already does placeholders": differentiate on mint source, unforgeability, destination binding, lifecycle, OTP evaluation, and threat model.
- "This is Autofill": Autofill is app-initiated and trusts the app; show it failing on agent flows.
- Post-entry re-exposure: the resolved value appears in the next frame; the redactor must work and be measured, or the contribution is undercut.
- WebView-origin binding granularity may be coarse; evaluate web flows carefully.
- Injection could still make the agent trigger a legitimate-but-unwanted action (e.g., log into the attacker's account on a bound app); scope the claim.

17 Killer Experiment

Run the 2510.27140 OTP-forwarding injection against an open-source agent on a login-plus-OTP test app, first with the agent reading the code off the screen (baseline: it forwards the code), then with SecretHandle, where the OTP is a handle bound to the login app's package. Verify that (a) the raw code never appears in the agent's observation or prompt, (b) `ENTER(handle)` into the attacker's field is refused by the resolver, and (c) the legitimate login completes. If exposure and redirection both drop to zero while login succeeds, the mechanism is proven; the full paper then adds credentials, wallet, and the redactor.

18 Publication Readiness

- ACM MobiSys: 7/10. Crisp OS mechanism, real flows, clear attack story; narrower scope than the OS-wide ideas.
- ACM MobiCom: 6/10.
- ACM SenSys: 2/10.
- IEEE S&P: 7/10. Capability model and injection resistance.
- USENIX Security: 7/10. Strong fit.
- ACM CCS: 6/10.
- NDSS: 7/10.
- PETS/PoPETs: 5/10.